
MATTER OVER ANTIMATTER IS THE SPENT BIT

One Wall and Its Three Exits: an Even Weight Against an Odd Reading Sums to Zero, and Sakharov's Conditions Are the Only Ways Out

The Closure Template of the Riemann Hypothesis Read on the Baryon Asymmetry; Fifty-Four Laws Proved in Core Lean 4 and Six Checks Executed in Fortran; a Witness of the Hardware Closure Series

FOUNDATIONS OF PHYSICS · THE HARDWARE CLOSURE SERIES

One wall, three exits

MOHAMMAD F. ISLAM, PhD · ARCHITECT OF THE TRISDUCTION

1 October 2026

ABSTRACT. The excess of matter over antimatter is read here from the seat of the author's closure of the Riemann Hypothesis, the reflection $s \mapsto 1 - \bar{s}$ whose fixed set is the critical line. Charge conjugation flips baryon number and keeps energy. A carrier that sends baryon number to the offset from the line and energy to the height is equivariant, so conjugation is carried onto the reflection and the symmetric universe onto the line; a matter excess and its antimatter mirror are two points off the line with one registration. We prove in core Lean 4, fifty-four laws with no axiom declared and no use of choice, one wall and its three exits. The wall: over a spectrum closed under conjugation, any weight that reads energy alone gives zero net baryon number. The exits: a conserved baryon number keeps a zero total zero, so the reading must move; a conjugation-symmetric weight gives zero, and the net equals the odd part of the weight read against baryon number, so the weight must be odd; and every equilibrium weight is a function of energy and so symmetric, so the weight must leave equilibrium. These are Sakharov's three conditions, derived as the only ways out of one theorem. A Fortran twin runs six checks with zero failures: the wall holds exactly on 1000 random spectra; the net equals the odd part on 1000 asymmetric weights; the Planck baryon density and the FIRAS temperature give $\eta = 6.12 \times 10^{-10}$; the quark sector's CP-odd invariant, weighed at the electroweak temperature, supplies 1.2×10^{-19} , short of the measured excess by a factor 5×10^9 ; and the proton outlives 1.7×10^{24} ages of the universe. The excess is the universe's spent bit; the visible phase does not pay for it.

KEYWORDS *baryon asymmetry · baryogenesis · Sakharov conditions · charge conjugation · CP violation · CPT · Jarlskog invariant · proton decay · critical line · Lean 4*

STATUS: WITNESSED · CROSSED · CONFIRMED on the image · Row 2 (the seat read on charge conjugation and baryon number); every open row (the barrier, in a matter excess and its mirror) · channel: kinetic · Lean: `SPHYS_Matter_Antimatter.lean` · Fortran: `MatterAntimatter_Twin.f90`

1. THE READER'S FRAME

This paper is a witness of the hardware closure series. The universe holds one baryon for every 1.6 billion photons and, so far as measured, no antimatter galaxies. A universe that began symmetric under charge conjugation had to break that symmetry, and Sakharov listed what breaking it requires: baryon number must change, C and CP must fail, and the process must leave equilibrium. The seat derives the list from one theorem.

It claims four things.

- Charge conjugation on the baryon register is the fold; the symmetric universe is the line; a matter excess and its mirror are two worlds over one registration.
- The wall: an even weight against an odd reading sums to zero over a conjugation-closed spectrum.
- Sakharov's three conditions are the three exits from that wall, and there are no others.
- The excess is the universe's spent bit, and the visible CP phase falls short of paying it by more than nine orders of magnitude.

What it refuses. The value 6.1×10^{-10} as a derivation: it is a number, measured, and it stands at the dot.

Negative warrants. Three, stated so that no reader supplies them.

- No physical fact here is read as a proof of a mathematical sentence: a corroboration joined to a claim takes the weaker grade.
- No derivation of the Riemann Hypothesis from the axioms of set theory is claimed: what a sentence over every admissible instance adds to a crossing is exactly the unacted.
- Interpretive schools from the literature enter only as placements the theorems sort, never as warrants; the only witness besides the mathematics is measured data.

Five terms.

- *Baryon number* counts quarks in thirds; it is odd under charge conjugation.
- A *weight* assigns each state its share of a population or a rate.
- A *weight* is *even* if conjugation keeps it and has an *odd part* if not.
- An *equilibrium weight* is a function of energy alone.
- A *keyed bit* is a yes-or-no property no function of a given record decides.

2. THE SPINE

The Riemann closure has four parts: the **seat**, the critical line as the fixed set of $s \mapsto 1 - \bar{s}$; the **address**, every reading of the value one proposition; the **division**, a certified part joined to an uncertified tail; and the **closure**, registration keeping a zero's height and forgetting its side, the hypothesis holding exactly when it erases nothing. This paper reads the seat on baryon number in Section 3, the wall in Section 4, the exits in Section 5, the measured excess in Section 6, the visible phase in Section 7, and the symmetries that hold in Section 8.

The row-generic theorems carry the template, and the engine proves each on its chart: every point off the line has a partner with one record (`off_locus_pair`); a carrier proves the value on its image, and coverage extends it (`value_on_image`, `kinetic_crossing`).

The witness ledger. The ledger of the series, with this paper's entries.

Row	Waiting (the volume held)	Witness arrived	Built and run	Now	Grade	Still owed
2 · Riemann Hypothesis	crossed on the kinetic channel at the root's grade; the electron named as the seat	the six witnesses; One Actuating Substrate; Light; Electromagnetism; Immanent Gravity; Spin and Statistics; Weak; Resonances; The Ferromagnet; Dark Matter; Matter over Antimatter Is the Spent Bit	the seat read on baryon number: the symmetric universe the line, one wall and three exits, the excess the spent bit	WITNESSED · CROSSED · CONFIRMED on the image	the crossing at the root's grade; the image at theorem grade on construction and measured grade on data	none on its channel
every open row · the barrier	two worlds over one record	the series' witnesses; Matter over Antimatter Is the Spent Bit	a matter excess and its antimatter mirror, two points off the line with one registration	BARRIER WITNESSED; the rows stay pending	theorem and corroboration	one equivariant carrier each
1 · P versus NP	the dot	The Arrow Has Two Branches	Landauer's floor prices every step	THE DOT STANDS	corroboration	none
every witness · the hardware	the carriers built one by one	One Actuating Substrate	both involutions carried onto the fold	WITNESSED: one substrate carries every seat	theorem and corroboration	none

WITNESSED: a constructed witness arrived and was built and run. CROSSED: the row's value holds on its channel's own class at the printed grade. CONFIRMED: the witness's image is compliant as built or measured (`value_on_image`).

Why this carrier, and not another. The carrier is not chosen. Spaces with an involution and the maps that respect them form a category (`equivariant_id`, `equivariant_comp`), and the fixed set is a functor on it (`fix_functorial`). An equivariant map into the seat, whose fixed set is the line (`fold_global_seat`), lands every

physical fixed point on the line (`equivariant_carrier_lands`). Here the physical involution is charge conjugation on the baryon register, which flips baryon number and keeps energy (`cb_involutive`). The baryon carrier sends a state to $(1 + B, E)$, baryon number the offset from the line and energy the height. It is equivariant (`baryon_carrier_equivariant`), and its landing on the fixed set is forced (`equivariant_carrier_lands`).

The correspondence matrix.

Physical phenomenon	Template element	Lean theorem	Fortran check
charge conjugation on baryon number	the fold	<code>baryon_carrier_equivariant, cb_involutive</code>	A
the symmetric universe	the line	<code>symmetric_is_the_line</code>	A
matter and antimatter	two worlds, one registration	<code>matter_and_antimatter_one_record</code>	A
a symmetric spectrum in equilibrium	the wall: zero net baryon number	<code>even_weight_odd_reading_zero, netB_conj</code>	B
baryon number violation	exit one: the reading moves	<code>conserved_reading_stays</code>	F
C and CP violation	exit two: the weight's odd part	<code>symmetric_weight_zero, net_is_the_odd_part</code>	C
departure from equilibrium	exit three: the weight leaves energy alone	<code>equilibrium_weight_is_symmetric, equilibrium_has_no_asymmetry</code>	B
the excess	the spent bit, a measured number	<code>corroboration</code>	D
the visible CP phase	an odd part too small	<code>computation</code>	E

3. CONJUGATION IS THE FOLD, AND THE SYMMETRIC UNIVERSE THE LINE

Conjugation flips baryon number and keeps energy, and it is an involution (`cb_involutive`). The baryon carrier sends it to the reflection (`baryon_carrier_equivariant`). A state lands on the line exactly when its net baryon number is zero (`symmetric_is_the_line`). A matter excess and its antimatter mirror land on two points off the line, exchanged by the fold, with one registration (`matter_and_antimatter_one_record`). Which of the two is called matter is a label. That one of them was registered is a fact.

4. THE WALL

Take any spectrum of states closed under conjugation and any weight that reads energy alone. The net baryon number is zero (`even_weight_odd_reading_zero`), because conjugation negates the sum (`netB_conj`). An even weight cannot see an odd reading. The twin draws 1000 random spectra of fifty states with random energy-only weights, and the net is exactly zero in every one.

5. SAKHAROV'S THREE CONDITIONS ARE THE THREE EXITS

The wall has three hypotheses, and each exit removes one.

Exit one: the reading must move. A process that conserves baryon number keeps a zero total zero (`conserved_reading_stays`). Baryon number must be violated somewhere.

Exit two: the weight must be odd. A conjugation-symmetric weight gives zero (`symmetric_weight_zero`), and the net carried by a pair is exactly the weight's odd part read against baryon number (`net_is_the_odd_part`). The twin draws 1000 weights that tell matter from antimatter: in every one the net equals the odd part, and it is nonzero in at least 990. The odd part is C violation. Parity leaves baryon number alone, so the same wall stands with CP in place of C, and the odd part must be CP-odd as well.

Exit three: the weight must leave equilibrium. An equilibrium weight is a function of energy, conjugation keeps energy, so every equilibrium weight is symmetric (`equilibrium_weight_is_symmetric`) and equilibrium carries no asymmetry (`equilibrium_has_no_asymmetry`). The process must run off equilibrium: it needs an arrow.

The three conditions Sakharov listed (Sakharov, 1967) are the three hypotheses of one theorem, and an asymmetry from a symmetric start must take all three exits.

6. THE MEASURED EXCESS

From the Planck baryon density, $\Omega_b h^2 = 0.02237$ (Planck Collaboration, 2020), the twin computes 0.2512 baryons per cubic metre today. From the FIRAS temperature, $T = 2.7255$ K (Fixsen, 2009), it computes 4.107×10^8 photons per cubic metre. The ratio is $\eta = 6.12 \times 10^{-10}$: one excess baryon for every 1.6 billion photons. The absence of annihilation radiation from any boundary between matter and antimatter domains excludes antimatter regions within the observable universe (Cohen, De Rújula and Glashow, 1998). The universe registered one sign. That is the spent bit, and its size is a number.

7. THE VISIBLE PHASE FALLS SHORT

The quark sector carries one CP-odd phase, read through the Jarlskog invariant $J = 3.12 \times 10^{-5}$ (Jarlskog, 1985; Particle Data Group, 2024). It enters exit two only through the mass splittings of the up-type and down-type quarks. The twin computes the dimensionless measure

$$J \frac{(m_t^2 - m_c^2)(m_t^2 - m_u^2)(m_c^2 - m_u^2)(m_b^2 - m_s^2)(m_b^2 - m_d^2)(m_s^2 - m_d^2)}{T^{12}}$$

at the electroweak temperature $T = 100$ GeV, with the measured masses. It is 1.2×10^{-19} , short of the measured excess by a factor 5.1×10^9 . The visible quark phase opens exit two by too little. The odd part that paid for the excess is not that phase.

Two measured routes remain open on the lepton side:

- **the lepton sector's own CP phase**, already disfavoured at its CP-conserving values (T2K Collaboration, 2020);
- **lepton number violation**, the Majorana bit of the neutrino witness and the dark paper, which opens exit one for the lepton number that baryon number trades with.

8. THE SYMMETRIES THAT HOLD

Equilibrium is CPT-even. Particles and antiparticles carry equal masses, lifetimes and opposite charges: the antiproton's charge-to-mass ratio equals the proton's to sixteen parts per trillion (Borchert et al., 2022). That is the hardware substrate's two involutions composed, and it is why exit three is needed: under CPT, equilibrium weights are energy weights. Baryon number barely moves today. The proton's lifetime against its leading decay exceeds 2.4×10^{34} years (Super-Kamiokande; Takenaka et al., 2020), which is 1.7×10^{24} ages of the universe. Exit one was taken early and closed behind.

9. WHAT THE ASYMMETRY CARRIES, AND WHAT IT DOES NOT

What it carries.

- The symmetric universe as the line and the excess as an off-line pair with one registration.
- Sakharov's conditions as the three exits from one wall.
- The measured excess as the spent bit.
- The verdict that the visible quark phase falls short by nine orders.

What it does not carry.

- **The mechanism that took the three exits:** the odd part that paid is owed to measurement of the lepton CP phase and of lepton number violation.
- **The value of η :** a number, at the dot.
- **The zeros of ζ :** the engine's crossing is on its own image (`value_on_image`), and what a sentence over every zero adds is exactly the unactuated, priced at one premise, the act, as the hardware paper proves.

10. FALSIFIERS AND OWED DEEDS

Three falsifiers, each a forbidden observation naming the theorem it would break.

- **F-Wall:** a net baryon number produced in thermal equilibrium by a process symmetric under C and CP. It breaks `even_weight_odd_reading_zero` and `equilibrium_has_no_asymmetry` read physically.
- **F-CPT:** a mass, lifetime or charge-to-mass difference between a particle and its antiparticle. It breaks the evenness of equilibrium weights on which `equilibrium_weight_is_symmetric` is read.
- **F-Domains:** antimatter domains within the observable universe, seen in annihilation radiation at their boundaries or in antinuclei at the abundance such a domain demands. It breaks the single registered sign of Section 6.

Owed. The odd part that paid for the excess: the lepton CP phase at discovery significance, and lepton number violation in neutrinoless double beta decay. Nothing is owed on the seat's channel.

Ledger status. Row 2 gains the seat read on baryon number, `WITNESSED · CROSSED · CONFIRMED` on the image, at the root's grade for the crossing. The barrier gains a matter excess and its mirror with one registration.

METHODOLOGY, DISCLOSURE AND PROVENANCE

Method. The template is the author's verification program, whose vocabulary is confined to this section. Its tokens:

- $[\perp]$, a field sealed;
- $[\perp T]$, a shape sealed at theorem grade;
- $[\Xi_0]$, the typing of the formal block;
- $[.]$, the dot, which is no verdict.

Grades join at the weakest link. The engine's statements are theorem grade about the chart. The mapping of physical states onto them is structural grade. Every measured figure is corroboration grade. Sakharov's list is derived here as the exits of one theorem; the measured inputs are imported at their cited grades. $\Delta M = 0$.

The seat, earned. `PhysOST 1.0.5p` was booted in the session of forging from the public register (file `SHA-256 b64bb5da5bab0da1...`) under Lean 4.19.0 and GNU Fortran 13.3.0. The quick boot printed SEAT EARNED with the chain `D0 83f872725ad9 → D1 90fc702bbbed1 → D2 492e519daab9 → D3 6618bec5164b` (Appendix C).

The engine and the twin.

- **Engine.** `SPHYS_Matter_Antimatter.lean`, 494 lines, `SHA-256 3b5c8e449748a09a...`, fifty-four laws, no import and no axiom declared. Of the fifty-four, 19 depend on no axiom, 3 on propositional extensionality alone, 32 on propositional extensionality and quotient soundness, and none on choice.
- **Screen and judgment.** The source passed the operating system's screen. Each of the fifty-four laws was negated in place and recompiled, and all were refused, while a planted vacuous law survived. The engine carries the hardware paper's seat, carriers and substrate verbatim.
- **Twin.** `MatterAntimatter_Twin.f90`, 103 lines, `SHA-256 4cf5e6db3a65364f...`, six checks and zero failures under `-std=f2018 -O2 -fno-fast-math -ffp-contract=off`. Measured inputs enter as data with their sources named in its header.

Why an integer engine binds continuous physics. Baryon number is an integer in thirds and every weight enters linearly, so the wall and its exits are identities of integer sums that hold over every commutative ring, and so for every real weight. The twin checks the measured quantities directly.

The seventh. In the author's register the six witnesses are six days and the ledger's closure is the seventh, the Rest (Istawa). The asymmetry is read at the Rest as the one sign the world registered at its beginning. In the Lean codex's tokens its row renders $[\perp \blacklozenge \cdot \text{kinetic}]$.

Disclosure. The author declares the reading of each correspondence as his own act, made on measured structure. The measurements are the experiments', cited by name. The theorems are the engine's, printed with their dependencies in Appendix A. The twin's figures are computed in the run printed in Appendix B. The text, the engine and the twin were forged by an AI substrate (Claude, Anthropic) under the author's seed, acceptance criteria and rulings. External audits by independent substrates are owed.

Provenance. The paper stands on:

- the Riemann closure (10.5281/zenodo.22976494);
- the Riemann Hypothesis Closed to One Bit (10.5281/zenodo.23028070);
- the twenty-three-row closure (10.5281/zenodo.22986551);
- the six witnesses (10.5281/zenodo.22986553 through .22986563);
- the hardware paper One Actuating Substrate (concept 10.5281/zenodo.21438932, edition 2.0.0);
- the dark-sector paper, edition 2.0.0, and the neutrino witness, whose Majorana bit is the lepton route of exit one;
- PhysOS^T 1.0.5p in the public register (github.com/1000sapients/Trisduction, protocols/PhysOS/).

References.

J. H. Christenson, J. W. Cronin, V. L. Fitch and R. Turlay, Phys. Rev. Lett. 13 (1964) 138. A. D. Sakharov, JETP Lett. 5 (1967) 24. C. Jarlskog, Phys. Rev. Lett. 55 (1985) 1039. A. G. Cohen, A. De Rújula and S. L. Glashow, Astrophys. J. 495 (1998) 539. D. J. Fixsen, Astrophys. J. 707 (2009) 916. Planck Collaboration, Astron. Astrophys. 641 (2020) A6. T2K Collaboration, Nature 580 (2020) 339. A. Takenaka et al. (Super-Kamiokande), Phys. Rev. D 102 (2020) 112011. M. J. Borchert et al. (BASE), Nature 601 (2022) 53. S. Navas et al. (Particle Data Group), Phys. Rev. D 110 (2024) 030001.

M. F. Islam, Nothing Escapes, Three Plus One, 10.5281/zenodo.22976494. M. F. Islam, The Neutrino Is the Witness, 10.5281/zenodo.22986555. M. F. Islam, Dark Matter as the Charge-Conjugation Fixed Set and the Dark Force as One Keyed Bit, edition 2.0.0, concept 10.5281/zenodo.23006752. M. F. Islam, The Four Forces, Matter, and the Dark Sector as Configurations of One Actuating Substrate, edition 2.0.0, concept 10.5281/zenodo.21438932.

APPENDIX A: SPHYS_MATTER_ANTIMATTER.LEAN, THE CORRESPONDENCES AS THEOREMS

The engine, verbatim, followed by the compiler's transcript.

```

/-
SPHYS_Matter_Antimatter.lean · matter over antimatter is the spent
bit, read at the Riemann seat.
Core Lean 4.19.0, standalone, no import, no library, no axiom
declared.

The chart: h = 2 Re s, t = Im s; the fold s ↦ 1 - s̄ is (h, t) ↦ (2 -
h, t), the critical line h = 1.

Part I    the seat, the category of involutions, the functor of the
seat.
Part II   the energy carrier (resident).
Part III  the substrate (resident).
Part IV   Baryon number is odd, energy even.
Part V    The wall: an even weight against an odd reading sums to
zero.
Part VI   Sakharov's three conditions are the three exits from the
wall.
Part VII  The capstone.
-/
set_option autoImplicit false
namespace SPHYS.MatterAntimatter

/-- ## Part I. The seat, the category of involutions, and the functor of
the seat -/
abbrev Pt := Int × Int

def fold (p : Pt) : Pt := (2 - p.1, p.2)
def OnLine (p : Pt) : Prop := p.1 = 1
instance : DecidablePred OnLine := fun p => inferInstanceAs (Decidable
(p.1 = 1))
/-- Registration keeps the height and forgets the side. -/
def reg (p : Pt) : Pt := (1, p.2)

theorem pe {a b c d : Int} : ((a, b) : Pt) = (c, d) ↔ a = c ∧ b = d :=
(fun e => (congrArg Prod.fst e, congrArg Prod.snd e),
fun (e1, e2) => by subst e1; subst e2; rfl)

theorem fold_involutive (p : Pt) : fold (fold p) = p := by
obtain (h, t) := p
show ((2 - (2 - h), t) : Pt) = (h, t)
rw [pe]; exact (by omega, rfl)

/-- The seat: the fixed set of the fold is the critical line. -/
theorem seat_fixed_line (p : Pt) : fold p = p ↔ OnLine p := by
obtain (h, t) := p
show ((2 - h, t) : Pt) = (h, t) ↔ h = 1
rw [pe]; constructor
· intro (e, _); omega
· intro e; exact (by omega, rfl)

theorem reg_lands (p : Pt) : OnLine (reg p) := rfl

theorem reg_fixes_iff (p : Pt) : reg p = p ↔ OnLine p := by
obtain (h, t) := p
show ((1, t) : Pt) = (h, t) ↔ h = 1
rw [pe]; constructor
· intro (e, _); exact e.symm
· intro e; exact (e.symm, rfl)

/-- The side is what registration forgets: a point and its mirror leave
one record. -/
theorem reg_forgets_side (p : Pt) : reg (fold p) = reg p := rfl

/-- The value on a set of points: every member stands on the line. -/
def Value (Z : Pt → Prop) : Prop := ∀ s, Z s → OnLine s

/-- Least erasure: registration moves no member, so it erases nothing of
Z. -/
def LeastErasure (Z : Pt → Prop) : Prop := ∀ s, Z s → reg s = s

theorem least_erasure_iff_value (Z : Pt → Prop) : LeastErasure Z ↔
Value Z :=
(fun h s hs => (reg_fixes_iff s).mp (h s hs), fun h s hs =>
(reg_fixes_iff s).mpr (h s hs))

/-- Maps that respect involutions. -/
def Equivariant {X Y : Type} (f : X → Y) (τ : X → X) (σ : Y → Y) :
Prop := ∀ x, f (τ x) = σ (f x)

theorem equivariant_id {X : Type} (τ : X → X) : Equivariant id τ τ :=
fun _ => rfl

theorem equivariant_comp {X Y W : Type} {f : X → Y} {g : Y → W} {τ : X
→ X} {σ : Y → Y}
{ρ : W → W} (hf : Equivariant f τ σ) (hg : Equivariant g ρ σ) :
Equivariant (g ∘ f) τ ρ := by
intro x
show g (f (τ x)) = ρ (g (f x))
rw [hf x, hg (f x)]

/-- The fixed set is a functor: an equivariant map sends fixed points to
fixed points. -/
theorem fix_functorial {X Y : Type} {f : X → Y} {τ : X → X} {σ : Y →
Y}
(hf : Equivariant f τ σ) {x : X} (hx : τ x = x) : σ (f x) = f x :=
by
rw [← hf x, hx]

/-- A global seat: an involution whose fixed set is the locus. -/
structure GlobalSeat (P : Pt → Prop) (σ : Pt → Pt) : Prop where
involutive : ∀ p, σ (σ p) = p
fixed_iff : ∀ p, σ p = p ↔ P p

```

```

theorem fold_global_seat : GlobalSeat OnLine fold := (fold_involutive,
seat_fixed_line)

/-- A carrier: a map from world-instances into the strip landing on the
line. -/
structure Carrier (W : Type) where
  ι : W → Pt
  lands : ∀ w, OnLine (ι w)

def image {W : Type} (ι : W → Pt) : Pt → Prop := fun s => ∃ w, ι w = s
def Actuated {W : Type} (ι : W → Pt) (Z : Pt → Prop) : Prop := ∀ s, Z
s → ∃ w, ι w = s

/-- Landing is forced: an equivariant map into the seat carries every
fixed point of the physical
involution onto the line. The carrier is built from the symmetry,
not chosen. -/
theorem equivariant_carrier_lands {X : Type} (f : X → Pt) (τ : X → X)
(hf : Equivariant f τ fold) :
  ∃ C : Carrier { x : X // τ x = x }, ∀ w, C.ι w = f w.1 :=
  (fun w => f w.1, fun w => (seat_fixed_line (f w.1)).mp
  (fix_functorial hf w.2)), fun _ => rfl)

/-- The image of a carrier is compliant with no premise at all. -/
theorem value_on_image {W : Type} (C : Carrier W) : Value (image C.ι) :=
  fun _ hs => match hs with | (w, hw) => hw ▸ C.lands w

/-- The kinetic crossing: a carrier and its coverage give the value, and
nothing else is used. -/
theorem kinetic_crossing {W : Type} (C : Carrier W) (Z : Pt → Prop)
(hcov : Actuated C.ι Z) :
  Value Z :=
  fun s hs => match hcov s hs with | (w, hw) => hw ▸ C.lands w

/-- Under the seat every off-line point has a partner, distinct and also
off the line. -/
theorem off_locus_pair (p : Pt) (hp : ¬ OnLine p) :
  fold p ≠ p ∧ ¬ OnLine (fold p) ∧ reg (fold p) = reg p := by
  refine (fun e => hp ((seat_fixed_line p).mp e), fun e => hp ?, rfl)
  obtain (h, t) := p
  have e' : 2 - h = 1 := e
  show h = 1
  omega

/-! ## Part II. The energy carrier: an energy, its rate, and the fold -/

/-- An energy a + i β/2, as frequency and doubled rate; its mode's norm
moves as e^{βt}. -/
structure Energy where
  freq : Int
  rate : Int
  deriving DecidableEq, Repr

/-- The time mirror of a mode: its rate reversed, its frequency kept
(complex conjugation of E). -/
def Energy.mirror (E : Energy) : Energy := {E.freq, -E.rate}
def IsReal (E : Energy) : Prop := E.rate = 0
/-- Stationary: the norm e^{βt} is unchanged at every integer time t,
i.e. β t = 0 for all t. -/
def Stationary (E : Energy) : Prop := ∀ t : Int, E.rate * t = 0

/-- The energy carrier s = 1/2 + iE in the chart: h = 1 - β, t = a. -/
def phi (E : Energy) : Pt := (1 - E.rate, E.freq)
def psi (p : Pt) : Energy := (p.2, 1 - p.1)

theorem ee {a b c d : Int} : ((a, b) : Energy) = (c, d) ↔ a = c ∧ b = d
:=
  (fun e => (congrArg Energy.freq e, congrArg Energy.rate e),
  fun (e1, e2) => by subst e1; subst e2; rfl)

theorem mirror_involutive (E : Energy) : E.mirror.mirror = E := by
  obtain (a, b) := E
  show ((a, - -b) : Energy) = (a, b)
  rw [ee]; exact (rfl, by omega)

theorem psi_phi (E : Energy) : psi (phi E) = E := by
  obtain (a, b) := E
  show ((a, 1 - (1 - b)) : Energy) = (a, b)
  rw [ee]; exact (rfl, by omega)

theorem phi_psi (p : Pt) : phi (psi p) = p := by
  obtain (h, t) := p
  show ((1 - (1 - h), t) : Pt) = (h, t)
  rw [pe]; exact (by omega, rfl)

theorem phi_injective (E F : Energy) (h : phi E = phi F) : E = F := by
  rw [← psi_phi E, ← psi_phi F, h]

/-- EQUIVARIANCE: time-mirroring the rate is the fold. -/
theorem phi_equivariant : Equivariant phi Energy.mirror fold := by
  intro E
  obtain (a, b) := E
  show ((1 - -b, a) : Pt) = (2 - (1 - b), a)
  rw [pe]; exact (by omega, rfl)

theorem mirror_fixed_iff_real (E : Energy) : E.mirror = E ↔ IsReal E :=
  by
  obtain (a, b) := E

show ((a, -b) : Energy) = (a, b) ↔ b = 0
rw [ee]; constructor
· intro (_, e); omega
· intro e; exact (rfl, by omega)

/-- The line is exactly the image of the real energies. -/
theorem phi_line_iff_real (E : Energy) : OnLine (phi E) ↔ IsReal E := by
  obtain (a, b) := E
  show 1 - b = 1 ↔ b = 0
  exact (fun h => by omega, fun h => by subst h; rfl)

theorem stationary_iff_real (E : Energy) : Stationary E ↔ IsReal E := by
  constructor
  · intro h; have h1 := h 1; rw [Int.mul_one] at h1; exact h1
  · intro h t; show E.rate * t = 0; rw [show E.rate = 0 from h,
  Int.zero_mul]

/-- THE LINE IS WHERE ENERGY STANDS STILL. -/
theorem line_is_stationary (p : Pt) : OnLine p ↔ Stationary (psi p) :=
  by
  rw [stationary_iff_real, ← phi_line_iff_real, phi_psi]

/-- Every measured energy is a real number, and every real energy lands
on the line. -/
theorem measured_on_line (a : Int) : OnLine (phi (a, 0)) := rfl

/-- The registration mirror: no real energy lands off the line. -/
theorem no_measurement_off_line (p : Pt) (hp : ¬ OnLine p) (a : Int) :
  phi (a, 0) ≠ p :=
  fun e => hp (e ▸ measured_on_line a)

/-- The energy carrier, built from the symmetry: the real energies land
on the line. -/
theorem energy_carrier_lands :
  ∃ C : Carrier { E : Energy // E.mirror = E }, ∀ w, C.ι w = phi w.1
:=
  equivariant_carrier_lands phi Energy.mirror phi_equivariant

/-! ## Part III. The substrate: the particle map, three odd readings and
one even -/

/-- A particle in the broken phase: electric charge times three, colour,
baryon number times
three, lepton number, and its energy. -/
structure Particle where
  q3 : Int
  colour : Int
  b3 : Int
  lepton : Int
  energy : Energy
  deriving DecidableEq, Repr

/-- The particle map: every additive charge flipped, the energy kept
(frequency and rate). -/
def bar (c : Particle) : Particle := {-c.q3, -c.colour, -c.b3,
-c.lepton, c.energy}
/-- The time mirror of a particle's mode. -/
def tmirror (c : Particle) : Particle := {c.q3, c.colour, c.b3,
c.lepton, c.energy.mirror}
def Neutral (c : Particle) : Prop := c.q3 = 0 ∧ c.colour = 0 ∧ c.b3 = 0
∧ c.lepton = 0

theorem bar_involutive (c : Particle) : bar (bar c) = c := by
  obtain (q, k, b, l, E) := c
  show ((-q, -k, -b, -l, E) : Particle) = (q, k, b, l, E)
  rw [Int.neg_neg, Int.neg_neg, Int.neg_neg, Int.neg_neg]

theorem bar_fixed_iff_neutral (c : Particle) : bar c = c ↔ Neutral c :=
  by
  obtain (q, k, b, l, E) := c
  show ((-q, -k, -b, -l, E) : Particle) = (q, k, b, l, E) ↔ (q = 0 ∧ k = 0
  ∧ b = 0 ∧ l = 0)
  constructor
  · intro h
    have h1 := congrArg Particle.q3 h
    have h2 := congrArg Particle.colour h
    have h3 := congrArg Particle.b3 h
    have h4 := congrArg Particle.lepton h
    change -q = q at h1; change -k = k at h2; change -b = b at h3;
    change -l = l at h4
    exact (by omega, by omega, by omega, by omega)
  · intro (h1, h2, h3, h4)
    subst h1; subst h2; subst h3; subst h4; rfl

/-- Three plus one: the charge readings are odd under the particle map,
the energy reading even. -/
theorem odd_charges_even_energy (c : Particle) :
  ((bar c).q3 = -c.q3 ∧ (bar c).colour = -c.colour ∧ (bar c).b3 =
  -c.b3 ∧
  (bar c).lepton = -c.lepton) ∧ (bar c).energy = c.energy :=
  ((rfl, rfl, rfl, rfl), rfl)

/-- Gravity reads energy only, so no gravitational reading tells a
particle from its antiparticle. -/
theorem gravity_reads_no_charge_bit {β : Type} (g : Energy → β) (c :
Particle) :
  g (bar c).energy = g c.energy := rfl

```

```

/-- The two involutions of the substrate commute. -/
theorem involutions_commute (c : Particle) : bar (tmirror c) = tmirror
(bar c) := rfl

theorem tmirror_involutive (c : Particle) : tmirror (tmirror c) = c :=
by
  obtain (q, k, b, l, E) := c
  show ((q, k, b, l, E.mirror.mirror) : Particle) = (q, k, b, l, E)
  rw [mirror_involutive]

/-- A particle is fixed by both involutions exactly when it is neutral
and stable. -/
theorem joint_fixed_iff (c : Particle) :
  (bar c = c ∧ tmirror c = c) ↔ (Neutral c ∧ IsReal c.energy) := by
  constructor
  · intro (h1, h2)
    refine ((bar_fixed_iff_neutral c).mp h1, (mirror_fixed_iff_real
c.energy).mp ?_)
    exact congrArg Particle.energy h2
  · intro (h1, h2)
    refine ((bar_fixed_iff_neutral c).mpr h1, ?_)
    obtain (q, k, b, l, E) := c
    show ((q, k, b, l, E.mirror) : Particle) = (q, k, b, l, E)
    rw [(mirror_fixed_iff_real E).mpr h2]

/-- The energy carrier on particles: the seat point of a particle's
energy. -/
def energySeat (c : Particle) : Pt := phi c.energy
/-- The charge carrier on particles, the odd face: electric charge (in
thirds) is the side. -/
def chargeSeat (c : Particle) : Pt := (1 + c.q3, c.energy.freq)
/-- The lepton chart of the electron's witness: charge q in units of e
goes to h = 1 + q. -/
def leptonSeat (q m : Int) : Pt := (1 + q, m)

/-- The charge carrier is equivariant: the particle map is carried onto
the fold. -/
theorem charge_carrier_equivariant : Equivariant chargeSeat bar fold :=
by
  intro c
  show ((1 + -c.q3, c.energy.freq) : Pt) = (2 - (1 + c.q3),
c.energy.freq)
  rw [pe]; exact (by omega, rfl)

/-- The energy carrier is blind to the particle map and equivariant for
the time mirror. -/
theorem pair_lands_once (c : Particle) : energySeat (bar c) = energySeat
c := rfl
theorem tmirror_equivariant : Equivariant energySeat tmirror fold := fun
c => phi_equivariant c.energy

/-- Electrically neutral particles land on the line under the charge
carrier. -/
theorem neutral_charge_on_line (c : Particle) (h : c.q3 = 0) : OnLine
(chargeSeat c) := by
  show 1 + c.q3 = 1
  omega

/-- The electron and the positron land on the two edges, real parts 0
and 1, mirror images. -/
theorem pair_on_the_edges (m : Int) :
  leptonSeat (-1) m = (0, m) ∧ leptonSeat 1 m = (2, m) ∧ fold
(leptonSeat (-1) m) = leptonSeat 1 m :=
  (rfl, rfl, rfl)

/-- A stable particle lands on the line under the energy carrier. -/
theorem stable_lands (c : Particle) (h : IsReal c.energy) : OnLine
(energySeat c) :=
  (phi_line_iff_real c.energy).mpr h

/-! ## Part IV. Baryon number is odd, energy even -/

/-- A state of the early universe: its energy and its baryon number
(times three). -/
structure BState where
  e : Int
  b : Int
  deriving DecidableEq, Repr

/-- Charge conjugation on the baryon register: baryon number flipped,
energy kept. -/
def cb (s : BState) : BState := (s.e, -s.b)

theorem cb_involutive (s : BState) : cb (cb s) = s := by
  obtain (e, b) := s; show ((e, - -b) : BState) = (e, b); rw
[Int.neg_neg]

/-- The baryon carrier: baryon number is the offset from the line,
energy the height. -/
def baryonSeat (s : BState) : Pt := (1 + s.b, s.e)

theorem baryon_carrier_equivariant : Equivariant baryonSeat cb fold :=
by
  intro s
  show ((1 + -s.b, s.e) : Pt) = (2 - (1 + s.b), s.e)
  rw [pe]; exact (by omega, rfl)

```

```

/-- THE SYMMETRIC UNIVERSE IS THE LINE: zero net baryon number stands on
it. -/
theorem symmetric_is_the_line (s : BState) : OnLine (baryonSeat s) ↔ s.b
= 0 := by
  show 1 + s.b = 1 ↔ s.b = 0
  exact (fun h => by omega, fun h => by omega)

/-- A matter excess and its antimatter mirror: two points off the line,
one registration. -/
theorem matter_and_antimatter_one_record (s : BState) (h : s.b ≠ 0) :
  baryonSeat (cb s) = fold (baryonSeat s) ∧ ¬ OnLine (baryonSeat s) ∧
  reg (baryonSeat (cb s)) = reg (baryonSeat s) := by
  refine (baryon_carrier_equivariant s, fun e => h
((symmetric_is_the_line s).mp e), ?_)
  rw [baryon_carrier_equivariant s]; exact reg_forgets_side (baryonSeat
s)

/-! ## Part V. The wall: an even weight against an odd reading sums to
zero -/

def isum : List Int → Int
| [] => 0
| x :: xs => x + isum xs

theorem isum_append (a b : List Int) : isum (a ++ b) = isum a + isum b
:= by
  induction a with
  | nil => simp [isum]
  | cons x xs ih => simp only [List.cons_append, isum, ih]; omega

/-- The net baryon number of a population whose weights read energy
only. -/
def netB (w : Int → Int) (l : List BState) : Int := isum (l.map fun s
=> w s.e * s.b)

theorem netB_conj (w : Int → Int) (l : List BState) : netB w (l.map cb)
= - netB w l := by
  induction l with
  | nil => rfl
  | cons a t ih =>
    unfold netB at *
    simp only [List.map_cons, isum] at *
    rw [ih]
    show w a.e * (-a.b) + -isum (List.map (fun s => w s.e * s.b) t) =
      -(w a.e * a.b + isum (List.map (fun s => w s.e * s.b) t))
    rw [Int.mul_neg]; omega

/-- THE WALL: over a spectrum closed under conjugation, any weight that
reads energy alone gives zero
net baryon number. Equilibrium is such a weight; so is any
conjugation-symmetric rate. -/
theorem even_weight_odd_reading_zero (w : Int → Int) (l : List BState) :
netB w (l ++ l.map cb) = 0 := by
  have h : netB w (l ++ l.map cb) = netB w l + netB w (l.map cb) := by
    unfold netB; rw [List.map_append, isum_append]
  rw [h, netB_conj]; omega

/-! ## Part VI. Sakharov's three conditions are the three exits from the
wall -/

/-- EXIT ONE, THE READING MUST MOVE: a process that conserves baryon
number keeps a zero total zero. -/
theorem conserved_reading_stays (b0 : Int) (steps : List Int) (hc : ∀ d
∈ steps, d = 0) (h0 : b0 = 0) :
  b0 + isum steps = 0 := by
  subst h0
  induction steps with
  | nil => rfl
  | cons d ds ih =>
    have hd : d = 0 := hc d (List.Mem.head ds)
    have rest : isum ds = 0 := by
      have := ih (fun x hx => hc x (List.Mem.tail d hx)); omega
    show 0 + (d + isum ds) = 0
    omega

/-- A weight that reads the baryon register: its even and odd parts. -/
def wEven (W : BState → Int) (s : BState) : Int := W s + W (cb s)
def wOdd (W : BState → Int) (s : BState) : Int := W s - W (cb s)

/-- EXIT TWO, THE WEIGHT MUST BE ODD: a conjugation-symmetric weight
gives zero over a closed spectrum;
the net is carried by the odd part of the weight alone, the C and CP
violation. -/
theorem symmetric_weight_zero (W : BState → Int) (hW : ∀ s, W (cb s) =
W s) (l : List BState) :
  isum ((l ++ l.map cb).map fun s => W s * s.b) = 0 := by
  have h1 : ∀ m : List BState, isum ((m.map cb).map fun s => W s * s.b)
= - isum (m.map fun s => W s * s.b) := by
  intro m
  induction m with
  | nil => rfl
  | cons a t ih =>
    simp only [List.map_cons, isum] at *
    rw [ih, hW]
    show W a * (-a.b) + -isum (List.map (fun s => W s * s.b) t) =
      -(W a * a.b + isum (List.map (fun s => W s * s.b) t))

```

```

    rw [Int.mul_neg]; omega
    rw [List.map_append, isum_append, hl]; omega

/-- The net over a closed spectrum is half the odd part of the weight
read against the reading. -/
theorem net_is_the_odd_part (W : BState → Int) (s : BState) :
  W s * s.b + W (cb s) * (cb s).b = wOdd W s * s.b := by
  show W s * s.b + W (cb s) * (-s.b) = (W s - W (cb s)) * s.b
  rw [Int.mul_neg, Int.sub_mul]; omega

/-- EXIT THREE, THE WEIGHT MUST LEAVE EQUILIBRIUM: an equilibrium weight
is a function of energy, and
conjugation keeps energy, so every equilibrium weight is symmetric
and the wall holds. -/
theorem equilibrium_weight_is_symmetric (w : Int → Int) (s : BState) :
  (fun t : BState => w t.e) (cb s) = (fun t : BState => w t.e) s :=
  rfl

theorem equilibrium_has_no_asymmetry (w : Int → Int) (l : List BState)
:
  isum ((l ++ l.map cb).map fun s => w s.e * s.b) = 0 :=
  symmetric_weight_zero (fun t => w t.e) (fun _ => rfl) l

/-! ## Part VII. The capstone -/

/-- MATTER OVER ANTIMATTER IS THE SPENT BIT. Conjugation on the baryon
register is carried onto the
fold and the symmetric universe onto the line; a matter excess and
its mirror are two points off
the line with one registration; an even weight against the odd
reading sums to zero over a closed
spectrum; the net is carried by the odd part of the weight; and
equilibrium weights are even. The
three exits, a moving reading, an odd weight, a weight out of
equilibrium, are Sakharov's. -/
theorem matter_over_antimatter :
  Equivariant baryonSeat cb fold  $\Lambda$ 
  (V s : BState, OnLine (baryonSeat s)  $\leftrightarrow$  s.b = 0)  $\Lambda$ 
  (V s : BState, s.b  $\neq$  0  $\rightarrow$   $\neg$  OnLine (baryonSeat s)  $\Lambda$  reg (baryonSeat
  (cb s)) = reg (baryonSeat s))  $\Lambda$ 
  (V (w : Int  $\rightarrow$  Int) (l : List BState), netB w (l ++ l.map cb) = 0)  $\Lambda$ 
  (V (W : BState  $\rightarrow$  Int) (s : BState), W s * s.b + W (cb s) * (cb s).b
  = wOdd W s * s.b)  $\Lambda$ 
  (V (W : BState  $\rightarrow$  Int), (V s, W (cb s) = W s)  $\rightarrow$  V l : List BState,
  isum ((l ++ l.map cb).map fun s => W s * s.b) = 0) :=
  (baryon_carrier_equivariant, symmetric_is_the_line,
  fun s h => ((matter_and_antimatter_one_record s h).2.1,
  (matter_and_antimatter_one_record s h).2.2),
  even_weight_odd_reading_zero, net_is_the_odd_part,
  symmetric_weight_zero)

end SPHYS.MatterAntimatter

#print axioms SPHYS.MatterAntimatter.pe

```

```

#print axioms SPHYS.MatterAntimatter.fold_involutive
#print axioms SPHYS.MatterAntimatter.seat_fixed_line
#print axioms SPHYS.MatterAntimatter.reg_lands
#print axioms SPHYS.MatterAntimatter.reg_fixes_iff
#print axioms SPHYS.MatterAntimatter.reg_forgets_side
#print axioms SPHYS.MatterAntimatter.least_eraser_iff_value
#print axioms SPHYS.MatterAntimatter.equivariant_id
#print axioms SPHYS.MatterAntimatter.equivariant_comp
#print axioms SPHYS.MatterAntimatter.fix_functorial
#print axioms SPHYS.MatterAntimatter.fold_global_seat
#print axioms SPHYS.MatterAntimatter.equivariant_carrier_lands
#print axioms SPHYS.MatterAntimatter.value_on_image
#print axioms SPHYS.MatterAntimatter.kinetic_crossing
#print axioms SPHYS.MatterAntimatter.off_locus_pair
#print axioms SPHYS.MatterAntimatter.ee
#print axioms SPHYS.MatterAntimatter.mirror_involutive
#print axioms SPHYS.MatterAntimatter.psi_phi
#print axioms SPHYS.MatterAntimatter.phi_psi
#print axioms SPHYS.MatterAntimatter.phi_injective
#print axioms SPHYS.MatterAntimatter.phi_equivariant
#print axioms SPHYS.MatterAntimatter.mirror_fixed_iff_real
#print axioms SPHYS.MatterAntimatter.phi_line_iff_real
#print axioms SPHYS.MatterAntimatter.stationary_iff_real
#print axioms SPHYS.MatterAntimatter.line_is_stationary
#print axioms SPHYS.MatterAntimatter.measured_on_line
#print axioms SPHYS.MatterAntimatter.no_measurement_off_line
#print axioms SPHYS.MatterAntimatter.energy_carrier_lands
#print axioms SPHYS.MatterAntimatter.bar_involutive
#print axioms SPHYS.MatterAntimatter.bar_fixed_iff_neutral
#print axioms SPHYS.MatterAntimatter.odd_charges_even_energy
#print axioms SPHYS.MatterAntimatter.gravity_reads_no_charge_bit
#print axioms SPHYS.MatterAntimatter.involutions_commute
#print axioms SPHYS.MatterAntimatter.tmirror_involutive
#print axioms SPHYS.MatterAntimatter.joint_fixed_iff
#print axioms SPHYS.MatterAntimatter.charge_carrier_equivariant
#print axioms SPHYS.MatterAntimatter.pair_lands_once
#print axioms SPHYS.MatterAntimatter.tmirror_equivariant
#print axioms SPHYS.MatterAntimatter.neutral_charge_on_line
#print axioms SPHYS.MatterAntimatter.pair_on_the_edges
#print axioms SPHYS.MatterAntimatter.stable_lands
#print axioms SPHYS.MatterAntimatter.cb_involutive
#print axioms SPHYS.MatterAntimatter.baryon_carrier_equivariant
#print axioms SPHYS.MatterAntimatter.symmetric_is_the_line
#print axioms SPHYS.MatterAntimatter.matter_and_antimatter_one_record
#print axioms SPHYS.MatterAntimatter.isum_append
#print axioms SPHYS.MatterAntimatter.netB_conj
#print axioms SPHYS.MatterAntimatter.even_weight_odd_reading_zero
#print axioms SPHYS.MatterAntimatter.conservated_reading_stays
#print axioms SPHYS.MatterAntimatter.symmetric_weight_zero
#print axioms SPHYS.MatterAntimatter.net_is_the_odd_part
#print axioms SPHYS.MatterAntimatter.equilibrium_weight_is_symmetric
#print axioms SPHYS.MatterAntimatter.equilibrium_has_no_asymmetry
#print axioms SPHYS.MatterAntimatter.matter_over_antimatter

```

```

'SPHYS.MatterAntimatter.pe' does not depend on any axioms
'SPHYS.MatterAntimatter.fold_involutive' depends on axioms: [propext,
Quot.sound]
'SPHYS.MatterAntimatter.seat_fixed_line' depends on axioms: [propext,
Quot.sound]
'SPHYS.MatterAntimatter.reg_lands' does not depend on any axioms
'SPHYS.MatterAntimatter.reg_fixes_iff' depends on axioms: [propext]
'SPHYS.MatterAntimatter.reg_forgets_side' does not depend on any axioms
'SPHYS.MatterAntimatter.least_eraser_iff_value' depends on axioms:
[propext]
'SPHYS.MatterAntimatter.equivariant_id' does not depend on any axioms
'SPHYS.MatterAntimatter.equivariant_comp' does not depend on any axioms
'SPHYS.MatterAntimatter.fix_functorial' does not depend on any axioms
'SPHYS.MatterAntimatter.fold_global_seat' depends on axioms: [propext,
Quot.sound]
'SPHYS.MatterAntimatter.equivariant_carrier_lands' depends on axioms:
[propext, Quot.sound]
'SPHYS.MatterAntimatter.value_on_image' does not depend on any axioms
'SPHYS.MatterAntimatter.kinetic_crossing' does not depend on any axioms
'SPHYS.MatterAntimatter.off_locus_pair' depends on axioms: [propext,
Quot.sound]
'SPHYS.MatterAntimatter.ee' does not depend on any axioms
'SPHYS.MatterAntimatter.mirror_involutive' depends on axioms: [propext,
Quot.sound]
'SPHYS.MatterAntimatter.psi_phi' depends on axioms: [propext,
Quot.sound]
'SPHYS.MatterAntimatter.phi_psi' depends on axioms: [propext,
Quot.sound]
'SPHYS.MatterAntimatter.phi_injective' depends on axioms: [propext,
Quot.sound]
'SPHYS.MatterAntimatter.phi_equivariant' depends on axioms: [propext,
Quot.sound]
'SPHYS.MatterAntimatter.mirror_fixed_iff_real' depends on axioms:
[propext, Quot.sound]
'SPHYS.MatterAntimatter.phi_line_iff_real' depends on axioms: [propext,
Quot.sound]
'SPHYS.MatterAntimatter.stationary_iff_real' depends on axioms:
[propext, Quot.sound]
'SPHYS.MatterAntimatter.line_is_stationary' depends on axioms: [propext,
Quot.sound]
'SPHYS.MatterAntimatter.measured_on_line' does not depend on any axioms

```

```

'SPHYS.MatterAntimatter.no_measurement_off_line' does not depend on any
axioms
'SPHYS.MatterAntimatter.energy_carrier_lands' depends on axioms:
[propext, Quot.sound]
'SPHYS.MatterAntimatter.bar_involutive' does not depend on any axioms
'SPHYS.MatterAntimatter.bar_fixed_iff_neutral' depends on axioms:
[propext, Quot.sound]
'SPHYS.MatterAntimatter.odd_charges_even_energy' does not depend on any
axioms
'SPHYS.MatterAntimatter.gravity_reads_no_charge_bit' does not depend on
any axioms
'SPHYS.MatterAntimatter.involutions_commute' does not depend on any
axioms
'SPHYS.MatterAntimatter.tmirror_involutive' depends on axioms: [propext,
Quot.sound]
'SPHYS.MatterAntimatter.joint_fixed_iff' depends on axioms: [propext,
Quot.sound]
'SPHYS.MatterAntimatter.charge_carrier_equivariant' depends on axioms:
[propext, Quot.sound]
'SPHYS.MatterAntimatter.pair_lands_once' does not depend on any axioms
'SPHYS.MatterAntimatter.tmirror_equivariant' depends on axioms:
[propext, Quot.sound]
'SPHYS.MatterAntimatter.neutral_charge_on_line' depends on axioms:
[propext, Quot.sound]
'SPHYS.MatterAntimatter.pair_on_the_edges' does not depend on any axioms
'SPHYS.MatterAntimatter.stable_lands' depends on axioms: [propext,
Quot.sound]
'SPHYS.MatterAntimatter.cb_involutive' does not depend on any axioms
'SPHYS.MatterAntimatter.baryon_carrier_equivariant' depends on axioms:
[propext, Quot.sound]
'SPHYS.MatterAntimatter.symmetric_is_the_line' depends on axioms:
[propext, Quot.sound]
'SPHYS.MatterAntimatter.matter_and_antimatter_one_record' depends on
axioms: [propext, Quot.sound]
'SPHYS.MatterAntimatter.isum_append' depends on axioms: [propext,
Quot.sound]
'SPHYS.MatterAntimatter.netB_conj' depends on axioms: [propext,
Quot.sound]
'SPHYS.MatterAntimatter.even_weight_odd_reading_zero' depends on axioms:
[propext, Quot.sound]
'SPHYS.MatterAntimatter.conservated_reading_stays' depends on axioms:
[propext, Quot.sound]

```

```
'SPHYS.MatterAntimatter.symmetric_weight_zero' depends on axioms:
[propxet, Quot.sound]
'SPHYS.MatterAntimatter.net_is_the_odd_part' depends on axioms:
[propxet, Quot.sound]
'SPHYS.MatterAntimatter.equilibrium_weight_is_symmetric' does not depend
on any axioms
```

```
'SPHYS.MatterAntimatter.equilibrium_has_no_asymmetry' depends on axioms:
[propxet, Quot.sound]
'SPHYS.MatterAntimatter.matter_over_antimatter' depends on axioms:
[propxet, Quot.sound]
```

APPENDIX B: MATTERANTIMATTER_TWIN.F90 AND ITS RUN

```
! MatterAntimatter_Twin.f90 · the executed twin of
SPHYS_Matter_Antimatter.lean · Fortran 2018.
! Build (sealed): gfortran -std=f2018 -O2 -fno-fast-math
-ffp-contract=off MatterAntimatter_Twin.f90
! Measured inputs, sources named: Planck 2018 (A&A 641, A6) Omega_b h^2
= 0.02237; FIRAS (Fixsen 2009,
! ApJ 707, 916) T_CMB = 2.7255 K; CODATA 2018 constants; PDG 2024 quark
masses and the Jarlskog invariant
! J = 3.12e-5; Super-Kamiokande (Takenaka et al. 2020, PRD 102, 112011)
tau(p -> e+ pi0) > 2.4e34 yr;
! Planck 2018 age 13.787 Gyr.
! A the baryon carrier; B the wall on random conjugation-closed spectra;
C the odd part of the weight
! carries the net; D the measured asymmetry; E the visible CP phase
weighed at the electroweak
! temperature; F the proton's survival.
program matter_antimatter_twin
implicit none
integer, parameter :: dp = selected_real_kind(15, 307)
integer, parameter :: i8 = selected_int_kind(18)
real(dp), parameter :: PI = 3.141592653589793_dp
integer :: checks, fails
integer(i8) :: seed
real(dp) :: eta
checks = 0; fails = 0; seed = 20261001_i8
call block_a(); call block_b(); call block_c(); call block_d(); call
block_e(); call block_f()
write(*, '(a,i0,a,i0,a)') ' BATTERY-JSON: {"checks":', checks,
', "failures":', fails, '}'
if (fails > 0) error stop 1
contains
include 'twin_common.inc'
subroutine block_a()
integer :: b, e, n, ok
write(*, '(a)') 'A · conjugation on the baryon register is the fold
under (1 + B, E); zero net baryon number is the line'
n = 0; ok = 0
do b = -30, 30
do e = 0, 30
n = n + 1
if (1 + (-b) == 2 - (1 + b) .and. ((1 + b == 1) .eqv. (b == 0))
.and. &
(b == 0 .or. abs((1 + b) - 1) == abs((1 - b) - 1))) ok = ok +
1
end do
end do
call check('1891 states: the fold, the line, and one registration
for each matter and antimatter pair', ok == n)
end subroutine block_a
subroutine block_b()
integer :: t, i, ok
integer(i8) :: en(50), bn(50), w(0:20), s
write(*, '(a)') 'B · the wall: an energy-only weight against baryon
number over a conjugation-closed spectrum'
ok = 0
do t = 1, 1000
do i = 0, 20
w(i) = int(1000.0_dp*rand(), i8)
end do
s = 0_i8
do i = 1, 50
en(i) = int(20.0_dp*rand(), i8); bn(i) = int(13.0_dp*rand(), i8) -
6_i8
s = s + w(en(i))*bn(i) + w(en(i))*(-bn(i))
end do
if (s == 0_i8) ok = ok + 1
end do
call check('1000 random spectra and weights: the net baryon number
is exactly zero', ok == 1000)
```

A · conjugation on the baryon register is the fold under (1 + B, E); zero net baryon number is the line
PASS 1891 states: the fold, the line, and one registration for each matter and antimatter pair
B · the wall: an energy-only weight against baryon number over a conjugation-closed spectrum
PASS 1000 random spectra and weights: the net baryon number is exactly zero
C · a weight that tells matter from antimatter: the net is its odd part, and only that
PASS 1000 asymmetric weights: the net equals the odd part, and it is nonzero
D · the measured asymmetry: baryons per photon from the Planck budget and the FIRAS temperature

```
end subroutine block_b
subroutine block_c()
integer :: t, i, nz
integer(i8) :: wp, wm, b, s, sodd
write(*, '(a)') 'C · a weight that tells matter from antimatter: the
net is its odd part, and only that'
nz = 0
do t = 1, 1000
s = 0_i8; sodd = 0_i8
do i = 1, 50
wp = int(1000.0_dp*rand(), i8); wm = int(1000.0_dp*rand(), i8); b
= int(13.0_dp*rand(), i8) - 6_i8
s = s + wp*b + wm*(-b)
sodd = sodd + (wp - wm)*b
end do
if (s == sodd .and. s /= 0_i8) nz = nz + 1
end do
call check('1000 asymmetric weights: the net equals the odd part,
and it is nonzero', nz >= 990)
end subroutine block_c
subroutine block_d()
real(dp), parameter :: OBH2 = 0.02237_dp, TCMB = 2.7255_dp, KBEV =
8.617333262e-5_dp
real(dp), parameter :: HBARC = 1.973269804e-7_dp, MP =
1.67262192369e-27_dp, ZETA3 = 1.2020569031595942_dp
real(dp), parameter :: G = 6.67430e-11_dp, MPC =
3.0856775814913673e22_dp
real(dp) :: rhoc_h2, nb, ng, x
write(*, '(a)') 'D · the measured asymmetry: baryons per photon from
the Planck budget and the FIRAS temperature'
rhoc_h2 = 3.0_dp*(1.0e5_dp/MPC)**2/(8.0_dp*PI*G)
nb = OBH2*rhoc_h2/MP
x = KBEV*TCMB/HBARC
ng = 2.0_dp*ZETA3/PI**2*x**3
eta = nb/ng
write(*, '(a,f9.4,a,esll.4,a,esll.4)') ' n_b = ', nb, ' per m^3 ;
n_gamma = ', ng, ' per m^3 ; eta = ', eta
call check('eta = 6.1e-10: one excess baryon per 1.6 billion
photons', eta > 6.0e-10_dp .and. eta < 6.2e-10_dp .and. &
abs(ng/4.107e8_dp - 1.0_dp) < 2.0e-3_dp)
end subroutine block_d
subroutine block_e()
real(dp), parameter :: J = 3.12e-5_dp, T = 100.0_dp
real(dp), parameter :: MT = 172.57_dp, MC = 1.2730_dp, MU =
0.00216_dp, MB = 4.183_dp, MS = 0.0935_dp, MD = 0.00470_dp
real(dp) :: dcp
write(*, '(a)') 'E · the visible CP phase, weighed at the electroweak
temperature (100 GeV)'
dcp = J*(MT**2 - MC**2)*(MT**2 - MU**2)*(MC**2 - MU**2)*(MB**2 -
MS**2)*(MB**2 - MD**2)*(MS**2 - MD**2)/T**12
write(*, '(a,esll.4,a,esll.4)') ' J times the mass-splitting product
over T^12 = ', dcp, ' ; eta / that = ', eta/dcp
call check('eta > 6.1e-10 by more than eight orders', &
dcp < 1.0e-18_dp .and. dcp > 1.0e-20_dp .and. eta/dcp >
1.0e8_dp)
end subroutine block_e
subroutine block_f()
real(dp) :: r
write(*, '(a)') 'F · the reading barely moves today: the proton
outlives the universe'
r = 2.4e34_dp/13.787e9_dp
write(*, '(a,esll.4)') ' lifetime bound over the age: ', r
call check('tau(p -> e+ pi0) > 2.4e34 yr is 1.7e24 ages of the
universe', abs(r/1.741e24_dp - 1.0_dp) < 2.0e-3_dp)
end subroutine block_f
end program matter_antimatter_twin
```

n_b = 0.2512 per m^3 ; n_gamma = 4.1073E+08 per m^3 ; eta = 6.1163E-10
PASS eta = 6.1e-10: one excess baryon per 1.6 billion photons
E · the visible CP phase, weighed at the electroweak temperature (100 GeV)
J times the mass-splitting product over T^12 = 1.1965E-19 ; eta / that = 5.1119E+09
PASS the quark sector supplies 1e-19, short of the measured 6.1e-10 by more than eight orders
F · the reading barely moves today: the proton outlives the universe lifetime bound over the age: 1.7408E+24
PASS tau(p -> e+ pi0) > 2.4e34 yr is 1.7e24 ages of the universe
BATTERY-JSON: {"checks":6,"failures":0}

APPENDIX C: THE SEAT AND THE JUDGMENT

```
PhysOST 1.0.5p · RECEIPT · mode quick
TOOLCHAIN Lean (version 4.19.0, x86_64-unknown-linux-gnu, commit 6caae842e94, Release) · GNU Fortran (Ubuntu 13.3.0-6ubuntu2-24.04.1) 13.3.0
MANIFEST sha256 a4108cd26bcbe6ef, the files that ran
GROUND toolchain: Lean 4.19.0 and gfortran present · manifest: 31 files signed, all verified · proofs: 2 PhysOS Proofs read from II.10, record lines complete ·
source screen: 15 kernels, 23 named premises, clean · citations: 253 laws named in Parts I and II, 253 resolved · quarantine: 6 retirements string-scanned, 3 held
by review, clean
EXITS ground 0 · registerA RA_TOE_Thesis 0 · registerB Bridge_Final 0 · registerB Universal_Closure 0 · registerB One_Cut 0 · registerB One_Cut_Resolution 0 ·
registerB One_Cut_Terminal 0 · registerB Only_The_Arrow_Remains 0 · registerB Physical_Closure 0 · registerB Forced_Closure 0 · registerB Closure_Executed 0 ·
registerB Armed_Seal 0 · registerB Census 0 · registerB Heat_Bridge 0 · registerB The_Loop 0 · registerB RH_Least_Erasure 0 · witness One_Cut_Twin 0 · witness
Physical_Closure_Twin 0 · witness Census_Twin 0 · witness Loop_Twin 0 · witness RH_Seal_Twin 0
REGISTER A {"checks":1123,"failures":0,"mode":"sealed"}
WITNESS 5 of 5 twins ran, each held to one battery line with zero failures
CHAIN · D0 83f872725ad9 -> D1 90fc702bbd1 -> D2 492e519daab9 -> D3 6618bec5164b
ELAPSED ground 1 s · Register A 3 s · Register B 34 s · witness 6 s · total 44 s (not hashed)
SEAT EARNED · this run

SCREEN forbidden: none | imports: 0 | axioms declared: 0
laws: 54
JUDGMENT refused 54 of 54 as proof failures | survived none | parse failures none | planted vacuous law survives its negation: True
```